

Padrões de Commit & Workflows de Git: O Guia de Sobrevivência em Equipe

Trabalhar sozinho com Git é fácil. O verdadeiro desafio surge quando você entra em uma equipe e precisa garantir que o histórico do projeto seja legível, organizado e que as tarefas de dezenas de desenvolvedores não se atropelem.

Neste guia, você aprenderá as melhores práticas do mercado para **comunicação através de commits** e os dois principais fluxos de trabalho (**Workflows**) do mundo real: **Git Flow** e **GitHub Flow**.

Parte 1: Padrões de Commit (Conventional Commits)

Imagine abrir o histórico de um projeto e ver a seguinte lista de mensagens:

- "ajustes"
- "consertando bug"
- "agora vai"
- "teste 123"

Isso não diz **nada** sobre o que realmente foi alterado. Para resolver esse caos, a comunidade criou o **Conventional Commits** (Commits Convencionais), uma especificação simples para criar mensagens de commit padronizadas e legíveis tanto por humanos quanto por ferramentas automatizadas (que podem gerar relatórios de novidades automaticamente).

A Estrutura de um Commit Perfeito

A estrutura básica de um commit padronizado segue este formato:

<tipo>(<escopo opcional>): <descrição curta>

[corpo opcional - explicação mais detalhada do motivo e impacto da mudança]

[rodapé opcional - ex: referências a tarefas do Jira, co-autores ou breaking changes]

Os Tipos de Commit Mais Comuns (Types)

O "tipo" indica **o que** você está fazendo no projeto. Aqui estão os principais tipos utilizados no mercado:

Tipo	O que significa?	Exemplo Prático
------	------------------	-----------------

 feat	Adição de uma nova funcionalidade (feature).	feat(auth): adiciona login social com Google
 fix	Correção de um bug (bug fix).	fix(checkout): corrige cálculo de frete grátis
 docs	Alteração exclusiva na documentação (ex: README, wikis).	docs: atualiza instruções de instalação no README
 style	Ajustes visuais, formatação de código ou espaçamento (sem alterar lógica).	style(cards): ajusta margens e cores dos botões
 refactor	Alteração no código que não corrige bug nem adiciona feature (melhorias internas).	refactor(api): otimiza queries de banco de dados
 test	Criação ou ajuste de testes automatizados.	test(login): adiciona teste unitário para e-mail inválido
 chore	Tarefas repetitivas ou de manutenção (ajustar dependências, webpack, build, etc).	chore(deps): atualiza versão do axios
 perf	Alteração de código com foco puramente em melhorar performance.	perf(imagens): implementa lazy loading de imagens

Breaking Changes (Alterações que Quebram Compatibilidade)

Se uma alteração que você fez vai quebrar o funcionamento de partes antigas do sistema (como mudar o nome de uma rota ou alterar um parâmetro obrigatório da API), você deve sinalizar isso adicionando uma exclamação (!) após o tipo e detalhando no rodapé:

feat(api)!: altera a url de autenticação de /v1/login para /v2/auth

BREAKING CHANGE: O endpoint antigo /v1/login foi totalmente descontinuado. Todos os clientes frontend precisam atualizar a URL padrão.

Parte 2: Git Flow (O Peso-Pesado Tradicional)

O **Git Flow** é um modelo de ramificação (branching) robusto criado por Vincent Driessen em 2010. Ele é focado em ciclos de lançamento de software (releases) bem definidos, estruturados e muito seguros.

A Analogia: A Fábrica de Carros Segura

Imagine uma montadora de carros:

- **main (O Showroom):** Aqui fica apenas o modelo final de luxo pronto para o cliente levar para casa. Intocado durante o dia a dia.
- **develop (A Linha de Montagem):** Onde os engenheiros testam peças novas e montam o chassi do carro base.
- **feature (O Laboratório de Peças):** Cada engenheiro trabalha isolado criando uma peça nova específica (ex: um motor elétrico). Quando a peça está pronta e testada, ela entra na linha de montagem (develop).
- **release (O Controle de Qualidade):** O carro está quase pronto, mas antes de ir para o showroom, passa por testes finos de pintura e polimento. Erros bobos achados aqui são corrigidos antes da venda.
- **hotfix (O Recall de Emergência):** O carro do showroom apresentou um vazamento crítico de óleo no motor do cliente. Um engenheiro de elite corrige isso na hora e aplica o conserto tanto no Showroom quanto na Linha de Montagem.

As Branches do Git Flow

No Git Flow, existem duas ramificações eternas e três temporárias:

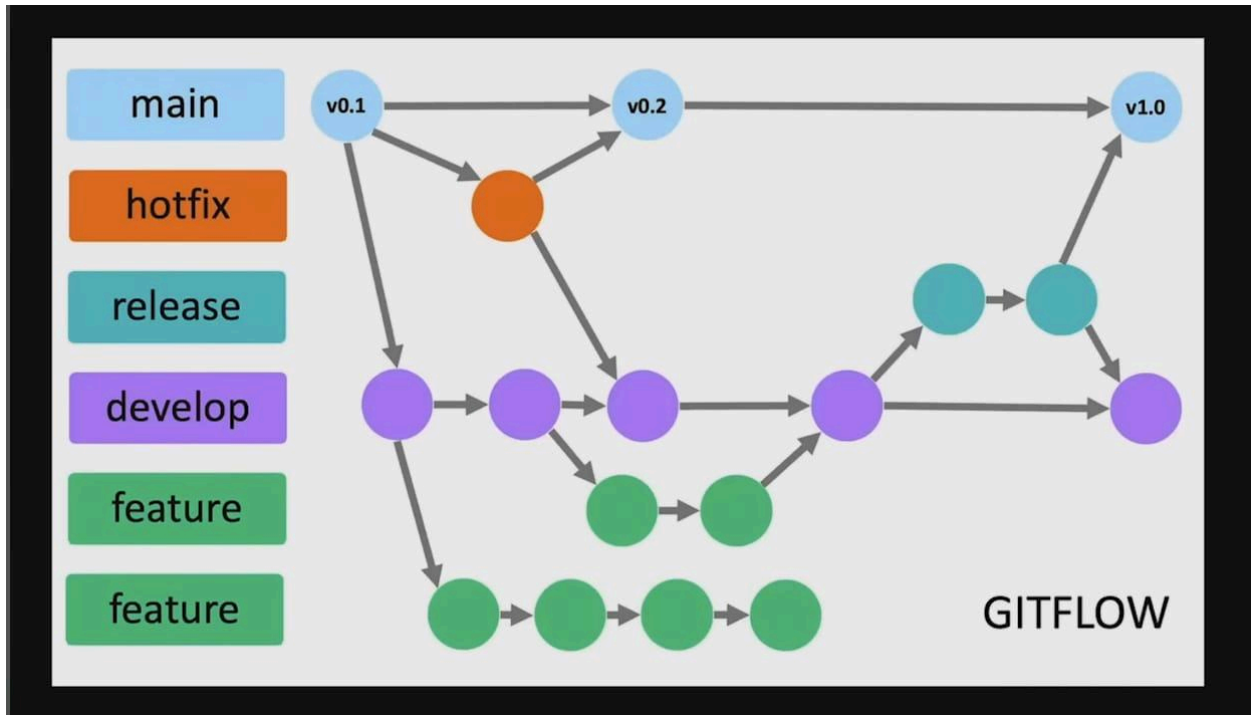
1. Branches Eternas (Nunca são deletadas):

- **main:** Armazena o código de produção. Cada commit nesta branch representa uma versão nova e estável do seu sistema lançada para os usuários.
- **develop:** Onde toda a integração de código acontece. É a branch "diária" dos desenvolvedores.

2. Branches Temporárias (Criadas e deletadas conforme a necessidade):

- **feature/*:** Criadas a partir de develop para trabalhar em novas tarefas. Ao terminar, fazem merge de volta na develop e o branch de feature é deletado.
- **release/*:** Criadas a partir de develop quando as tarefas do próximo lançamento estão prontas. Serve apenas para testes finais e pequenos ajustes de bugs de última hora. No final, faz merge na main (com tag de versão, ex: v1.2.0) e de volta na develop.
- **hotfix/*:** Criadas diretamente a partir de main em casos de extrema urgência em produção. Corrigem o problema e fazem merge imediatamente na main e na develop.

Visualização do Histórico do Git Flow



- **Vantagens:** Extremamente controlado, seguro para softwares de grande porte que exigem homologação rígida e lançamentos agendados.
- **Desvantagens:** Muito complexo para projetos pequenos ou rápidos. O histórico fica com muitos merges e branches ativas simultaneamente.

Parte 3: GitHub Flow (O Ágil e Moderno)

Criado pelo próprio GitHub, o **GitHub Flow** é um modelo muito mais simples, leve e focado no conceito de **Entrega Contínua (Continuous Delivery)**. Se o seu time coloca código em produção várias vezes ao dia, o Git Flow é lento demais para vocês; o GitHub Flow é a escolha ideal.

A Analogia: O Jornal Digital Contínuo

Ao contrário de um livro impresso que exige meses de revisão antes de ir para a gráfica (como o Git Flow), o GitHub Flow funciona como um portal de notícias em tempo real:

- A página principal (**main**) está sempre no ar e precisa estar 100% livre de erros.
- Um jornalista quer escrever uma notícia nova. Ele cria um rascunho em uma folha separada (**feature branch**).
- Quando termina de escrever, ele submete o texto ao editor-chefe para revisão através de um painel de comentários (**Pull Request**).

- O editor dá o feedback, corrige pequenos pontos na mesma hora e aprova o texto.
- O rascunho é adicionado ao portal e publicado imediatamente para o mundo inteiro (**Merge para a main e Deploy**).

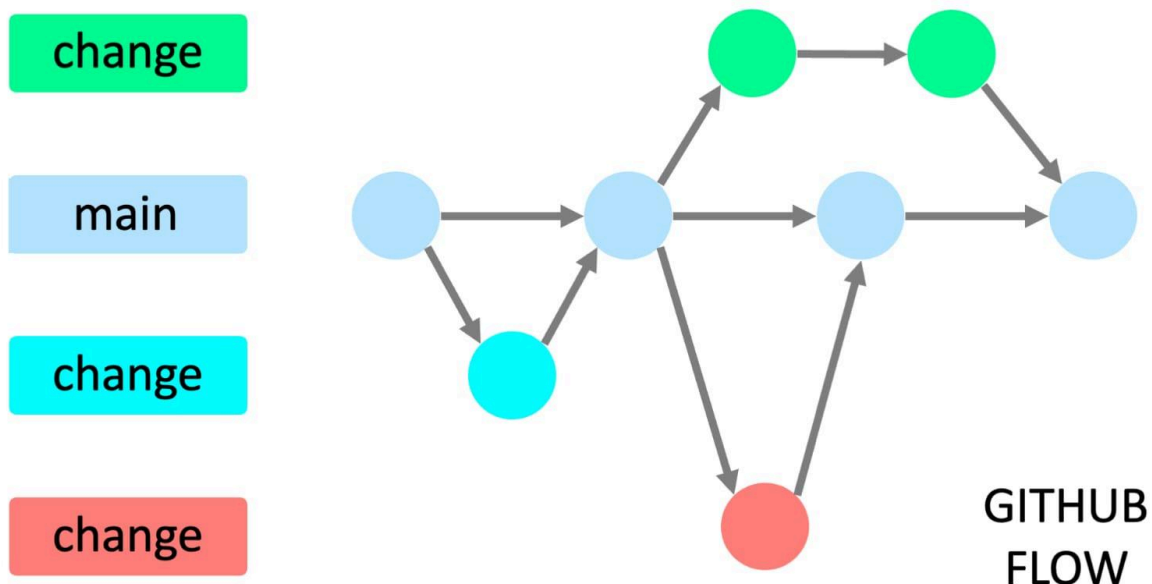
⚙️ As Regras de Ouro do GitHub Flow

O GitHub Flow possui apenas **6 regras de funcionamento**:

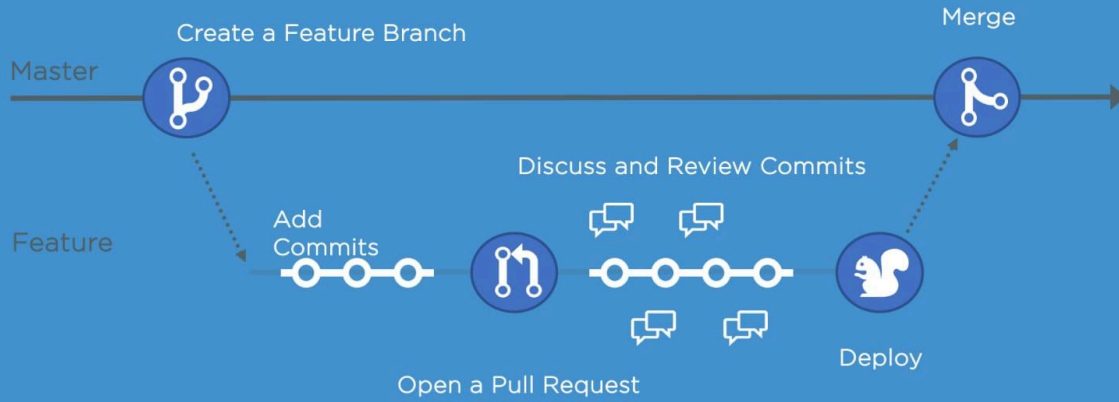
1. **Qualquer código na branch main deve estar pronto para produção:** Se alguém baixar a main agora e colocar no ar, ela **precisa** funcionar sem erros.
2. **Crie branches com nomes descritivos:** Crie sua ramificação a partir da main com nomes claros como feature/add-payment-method ou bugfix/broken-header.
3. **Faça Commits frequentes e empurre (push) para o servidor:** Salve seu trabalho na nuvem constantemente para que todos saibam no que você está trabalhando.
4. **Abra um Pull Request (PR):** Quer integrar seu código ou precisa de ajuda técnica? Abra um PR. O PR é uma sala de discussão focada naquele pedaço de código.
5. **Revisão de Código (Code Review):** Outros desenvolvedores lêem suas alterações, sugerem melhorias, os testes automatizados rodam e, quando tudo estiver aprovado, o PR recebe o "Green Light" (Sinal Verde).
6. **Merge e Deploy Imediato:** Assim que o PR for mesclado na main, o código deve ser implantado (deploy) em produção de forma automatizada (usando ferramentas de CI/CD).



Visualização do Histórico do GitHub Flow



GitHub Flow



- **Vantagens:** Extremamente simples, rápido, reduz o atrito burocrático e estimula o feedback rápido entre os desenvolvedores através dos Pull Requests.
- **Desvantagens:** Menos controle para sistemas legados que dependem de datas fixas de distribuição ou que necessitam de múltiplos ambientes de teste antes da produção.

Parte 4: Git Flow vs GitHub Flow (Qual escolher?)

A escolha do fluxo ideal depende inteiramente do produto que você está desenvolvendo e de como sua equipe trabalha.

Critério de Comparação	Git Flow	GitHub Flow
Complexidade	Alta (múltiplas branches com regras rígidas).	Baixa (apenas main e branches de tarefas).
Ambiente de Produção	Lançamentos planejados (semanais, mensais ou anuais).	Lançamento contínuo (múltiplos deploys diários).
Ideal para...	Softwares instaláveis (mobile, desktop, sistemas embarcados) ou produtos corporativos rígidos.	Web Apps, microserviços, SaaS e produtos web modernos hospedados na nuvem.

Foco de Revisão	Ramificações de release e homologação em equipe.	No debate ativo dentro do Pull Request .
Tamanho da Equipe	Funciona bem para grandes corporações integrando dezenas de times.	Perfeito para startups e equipes ágeis que precisam de velocidade.

Dominando os Padrões de Commit e escolhendo o Workflow adequado para a sua equipe, o desenvolvimento do seu software ganha velocidade, organização e, acima de tudo, paz de espírito! 🚀